

Write tests that catch bugs

Swap roles: write the tests yourself, and make them catch planted bugs.

The big idea

Until now, **the tests checked your code**. This time you **write the tests**, for a function someone else wrote.

A test says: "call this with **this**, expect **that** back". It's like `assert df["price"].min() >= 0` in a notebook, kept in its own file and re-run every time the code changes.

A test is only useful if it **fails when the code is wrong**. A smoke alarm that never beeps protects nothing.

The function you'll test

`make_slug(text)` lives in `slugs.py`, next to your file. You don't write or change it.

```
make_slug("hello world")           # "hello-world"
make_slug("  Hello, World!  ")     # "hello-world"      lower-case, ends trimmed
make_slug("Django & DRF 101")      # "django-drf-101"   a run of symbols = ONE "-"
make_slug("word " * 30)            # at most 50 characters, never ends with "-"
make_slug("!!!")                   # raises ValueError: nothing usable is left
```

How the grader runs your tests

Run 1: the correct make_slug

- The grader runs **all** your test methods
- Every one must **pass** (test 2)
- Fails here? Your expected value is wrong
- You need at least **4** test methods (test 1)

Runs 2-5: one planted bug each

- No lower-case: "Hello-World" (test 3)
- No collapse: "django---drf" (test 4)
- No length limit: 149 characters (test 5)
- No error: "!!!" gives "" (test 6)
- At least one of your tests must **fail**

New words

test	A small function that calls some code and checks the result.
unittest	Python's built-in testing library.
test case class	Inherits from <code>unittest.TestCase</code> ; each <code>test...</code> method is one test.
assertion	A check like <code>self.assertEqual(a, b)</code> . If it's false, the test fails.
bug / mutant	A copy of the function with one small mistake planted in it.
raise an error	The function stops and reports a problem, like <code>ValueError</code> .

Tool: a test case class

Inherit from `unittest.TestCase`; every method whose name starts with `test` is one test.

```
import unittest

class ShoutTests(unittest.TestCase):
    def test_makes_upper_case(self):
        result = "tea".upper()          # 1. call the code
        self.assertEqual(result, "TEA") # 2. check the answer
```

Tool: assertEquals(actual, expected)

Outside a class, borrow the checks from a plain `unittest.TestCase()` to try them in the Console.

```
import unittest

check = unittest.TestCase()
check.assertEqual("tea".upper(), "TEA")
print("first check passed")
check.assertEqual("tea".upper(), "Tea")
```

OUTPUT

```
first check passed
AssertionError: 'TEA' != 'Tea'
```

Tools: True/False, size and errors

`assertRaises` passes only if the code **inside** the `with` block raises that error.

```
check = unittest.TestCase()
name = "  Grace Hopper  "
check.assertTrue(name.startswith(" "))
check.assertFalse(name.endswith("r"))
check.assertLessEqual(len(name.strip()), 12)
with check.assertRaises(ValueError):
    int("twelve")
print("all four checks passed")
```

OUTPUT

```
all four checks passed
```

Inside vs outside the with block

Outside: the test crashes

- `make_slug("!!!")` is **not** indented under `with`
- The `ValueError` happens outside, so nothing catches it
- Correct code: the test **crashes** (test 2 fails)
- Fix: indent the call one level under `with`

Inside: the error is checked

- `make_slug("!!!")` **is** indented under `with`
- Error raised (correct code): the test **passes**
- No error (the bug): the test **fails**

Pattern: one test per behaviour

The same shape as your task, for a different function:

```
import unittest

def make_tag(text):
    tag = text.strip().lower().replace(" ", "_")
    if not tag:
        raise ValueError("empty tag")
    return tag

class MakeTagTests(unittest.TestCase):
    def test_lowercases(self):
        self.assertEqual(make_tag(" Data Science "), "data_science")
    def test_blank_raises(self):
        with self.assertRaises(ValueError):
            make_tag(" ")
```

Your task, step by step

- 1 Keep the example `test_basic_title`: it passes on the correct code → **tests 1-2**
- 2 Test capitals: " Hello, World! " should give "hello-world" → **test 3**
- 3 Test a run of symbols: "Django & DRF 101" gives "django-drf-101" → **test 4**
- 4 Long title "word " * 30: length at most 50, doesn't end with "- " → **test 5**
- 5 Put `make_slug("!!!")` inside `with self.assertRaises(ValueError):` → **test 6**

Common mistakes

- A method named `check_lowercase` never runs: names must start with `test`
- Two methods with the same name: only the last one counts
- Expecting `"Hello-World"`: run `make_slug(...)` in the Console first
- `make_slug("!!!")` outside the `with` block crashes the test
- `assertEqual(...)` without `self.` gives `NameError`

Where you'll use this

A teammate "tidies up" the blog's slug code and forgets `.lower()`. Without a test, links like `/blog/Hello-World/` go live. With your test, the change fails before anyone ships it.

- Django's `TestCase` builds on `unittest.TestCase`, with the same `assert...` methods
- Every exercise in this course is graded by tests written this way
- Real projects add a test for each endpoint, so changes can't quietly break it

Check yourself

1. Why does the example test with `"hello world"` catch none of the bugs?
2. Your tests pass on a buggy version. What does that tell you?
3. What happens if two test methods have the same name?
4. Why must `make_slug("!!!")` be indented under the `with` line?

Keep learning

[How to Write Unit Tests in Python – with Example Test Code](#)

freeCodeCamp · article · beginner. Start here: writes a TestCase for a small function using assertEquals and assertRaises, the same pattern as your slug tests.

[Writing your first Django app, part 5 \(testing\)](#)

Django docs · tutorial · beginner. Shows how to find a real bug, write a test that exposes it, then fix it: exactly the mindset this exercise trains.

[unittest: Unit testing framework](#)

Python docs · docs · intermediate. The official list of assert methods and how test discovery works; keep it open while writing tests.

[Writing and running tests](#)

Django docs · docs · intermediate. How Django builds on unittest with its own TestCase and test database, for when you test models and views.